

Individual Project Progress Report

(June 10 - July 15, 2026)

Name: Shakil Ahmed ID: 2221453042

CSE440.1 Artificial Intelligence Group 5

Project Overview

The course began on June 10, 2026. This report summarizes my individual contribution from the beginning of the course through July 15. Our project, **Autonomous Volcanic Terrain Exploration Using a Markov Decision Process (MDP)**, models a rover that must explore a hazardous grid while balancing coverage, scientific value, movement cost, and safety. My main responsibility was the project foundation, terrain-generation environment, and MDP core. I also supported later integration by reviewing branches, merging available code, and debugging compatibility problems.

Week 1 (Jun 10 - Jun 14)

During the opening week, I reviewed the course manual, semester-project requirements, GitHub expectations, update-report rules, and the MDP material needed for the project. The group had not yet finalized the topic, so I focused on understanding what a suitable AI project needed to demonstrate: clear states, actions, uncertainty, rewards, an optimal policy, testable code, and visible GitHub progress. I also noted the teacher's requirement that each member's update report must describe original, individual work week by week.

Week 2 (Jun 15 - Jun 21)

The four-person group was formed and we discussed several possible AI topics. We selected autonomous volcanic-terrain exploration because it naturally fits an MDP: the rover makes sequential decisions, movement can be uncertain, and each terrain type has a different safety cost or exploration value. We divided the work into MDP core, agent and simulation, visualization and demo outputs, and experiments and comparison.

I then converted the shared repository, which initially contained an empty README, into a structured Python project. I created `main.py`, `requirements.txt`, configuration and utility modules, and the required data, `support`, `outputs`, and others directories. I prepared separate module files so each member

could work without overwriting another member's responsibilities, and verified the starter project using `python main.py`.

Week 3 (Jun 22 - Jun 28)

I implemented `support/terrain.py` and created a reusable `Terrain` class. The world is represented as a configurable two-dimensional grid containing seven cell types. Seeded random generation allows the same terrain to be reproduced for testing, while command-line options allow the seed and grid size to be changed. The base station is protected from being overwritten, terrain-cell counts are reported, and generated maps can be saved to CSV files.

The terminal screenshots use compact characters to display the grid: `.` means a safe traversable cell, `L` means lava, `C` means crater, `G` means gas-emission zone, `R` means rock or obstacle, `S` means science point, and `B` means the base station. I connected the generator to `main.py`, which prints the grid, counts, legend, and output-file location.

Week 4 (Jun 29 - Jul 5)

I implemented the central AI component in `support/mdp.py`. Each valid non-rock coordinate is treated as a state. The action set contains UP, DOWN, LEFT, RIGHT, STAY, and SCAN. Directional movement is stochastic: the command may execute as intended, slip to either side, or leave the rover in place. Invalid moves and attempts to enter rocks keep the agent in the current state.

I designed the reward model to encourage useful exploration and science collection while assigning increasingly severe penalties to gas, craters, and lava. I then implemented value iteration and policy extraction so the system can estimate long-term state values and select the action with the greatest expected return.

```
State: valid (row, col), excluding ROCK
Actions: UP, DOWN, LEFT, RIGHT, STAY, SCAN
Transition probabilities: 0.75, 0.10, 0.10, 0.05
gamma = 0.92
Solution: value iteration + policy extraction
```

```

D:\Web Dev\Autonomous Volcanic Terrain Exploration Using Markov Decision Proc
=====
Project Title
=====
Autonomous Volcanic Terrain Exploration Using Markov Decision Process (MDP)
=====
Week 2: Volcanic Terrain Generation
=====
Seed: 42
Grid size: 10x10
=====
Generated Terrain Grid
=====
. L R G . . . L S L
G . L G . . G . . L
. . R C . R L L . .
. . . . R . . . . .
. L G R L G C R . R
R G R . . . C . C R
. . . . G L R R
G . . R . R . S R G
. R . R G . . L L
B C . . S L R . . .
=====
Terrain Cell Counts
=====
SAFE: 49
LAVA: 13
CRATER: 5
GAS: 11
ROCK: 18
SCIENCE: 3
BASE: 1

```

Figure 1: Default 10x10 terrain generated with seed 42, including the grid and terrain-cell counts.

```

D:\Web Dev\Autonomous Volcanic Terrain Exploration Using Markov Decision Process>
=====
Project Title
=====
Autonomous Volcanic Terrain Exploration Using Markov Decision Process (MDP)
=====
Week 2: Volcanic Terrain Generation
=====
Seed: 15
Grid size: 20x20
=====
Generated Terrain Grid
=====
. L C L . . . . G R . R R G . . G
. . G S S L . R . . . S . . S . L C C .
. . R S . L C L S . . S . C . R G R . R
. R S L . . S C S R . . . R L . . C S S
. L L . . S . . R L L R . R R . . R
. . . L . R R . C G . R . . R R . R
L C . . C . R . C L R . . S . R . L .
C . . . . R G . . C . S C . . R . .
. S R C R . L . . . L . G . . . . G C
. L S S . R . . S S S . S S . G L R
. C . . . S L S S . R . S . . C . S S .
. . G G . . . . R . S C . . C . R .
. C . G C . . . . R S . . S . S . .
R R . R . L C G . G . L . . R G R S G G
. C . . . . . . . . L . G R R . .
. R R . R L . . . . S L . S C . L R .
R G R . L C R . . . L . C . . S . .
R G . . . . S C G . . R . . C R G
C G S . R . R . . . C . C . C R G
B C . S C S . C . . . . R L R C . R .
=====
Terrain Cell Counts
=====

```

Figure 2: A 20x20 configuration test using --seed 15 --size 20, showing that the environment is configurable.

Character key used in both screenshots: . = safe cell; L = lava flow; C = crater; G = gas-emission zone; R = rock/obstacle; S = science point; B = base station.

Week 5 (Jul 6 - Jul 12)

I tested the terrain and MDP modules independently and reviewed their interfaces before the rest of the team connected agent, visualization, and experiment code. I confirmed reproducibility using fixed seeds, tested both 10x10 and 20x20 maps, verified CSV saving, and checked that the MDP state list excludes rocks. I also updated the project documentation and prepared a precise file-ownership and branch plan for the four members so that generated AI code would remain within scope and would not replace working core modules.

I documented the dependency order: the completed terrain and MDP core first, followed by agent and simulation, real-path visualization, and comparative experiments. Each member received a specific branch, assigned files, test commands, and separate Update 1 and Update 2 acceptance criteria.

Week 6 (Jul 13 - Jul 15) - Update 1 Complete

During the final update period, I helped integrate available member contributions into the main codebase. I reviewed changed files before merging, checked that modules imported the existing Terrain and VolcanicMDP classes instead of duplicating them, and debugged path, import, and interface mismatches. After merging, I reran the core commands and corrected documentation that no longer matched the implemented state of the project.

```

python main.py
python main.py --seed 15 --size 20
python support/terrain.py
python support/mdp.py

```

The commands completed successfully. The same seed produced consistent maps, the size argument generated the larger grid shown in Figure 2, CSV output was created, and the MDP module executed without errors.

My Personal Contribution (Summary)

I contributed to topic selection and technical planning; established the project repository and module structure; implemented the configurable, seeded volcanic-terrain generator and CSV persistence; defined MDP states, actions, stochastic transitions, and rewards; implemented value iteration and policy extraction; created the team file-ownership and branch strategy; and supported code merging and debugging during integration.

Current Status and Next Plan

The project foundation, terrain generator, and MDP core are operational. These modules now provide stable interfaces for simulation, visualization, and experiments. My next work is to continue reviewing merged code, verify that the final simulation follows the extracted policy, help debug end-to-end runs, and prepare the technical MDP explanation and verified results for the final report, presentation, and one-minute demo.

Development Tools and Attribution

The implementation was developed in Python and managed through GitHub. OpenAI Codex was used as a coding-assistance tool for scaffolding and review. I personally reviewed the changes, executed the modules, debugged integration issues, and verified the outputs described in this report.